

NAME :		DATE :	
SCHEDULE :		RATING :	

Laboratory 1: Basic I/O (Input/Output)

Status Check	Criteria	Poor 1	Fair 2	Good 3	Excellent 4
Advance (100 pts. plus 5 extra pts.)	Functionality	___ Not working	___ Several errors	___ Mostly work	___ Fully functional
	Design	___ Confusing or difficult to follow	___ Lacks clear structure.	___ Mostly organized	___ Content is well-arranged
On Time (80 pts to 100 pts)	Completeness	___ Incomplete Data	___ Partial Completion	___ Mostly complete	___ Fully complete
	Neatness of work	___ Messy and disorganized	___ Somewhat neat and good	___ Neat and organized	___ Exceptionally neat and organized
Late (70 pts to 79 pts)	Analytical Ability	___ Lack Analysis	___ Limited Analysis	___ Demonstrates good ability	___ Good analytical skills and ability

Objective:

Introduce the basic Input/Output programming using VB.Net and Java

Familiarize students with the development environment (Visual Studio for VB.NET, IntelliJ/Eclipse/NetBeans for Java).

Write a program using VB.Net and Java

Introduction:

All computer programs must interact with users in some way. This interaction is done through:

- Input: Getting data from the user (keyboard, file, mouse, etc.).
- Output: Displaying results back to the user (screen, console, file, etc.).

In console-based applications, input and output are performed using simple text commands.

VB.Net Output

Console.WriteLine() → Displays text and moves to the next line.

Console.Write() → Displays text but stays on the same line.

Example:

Console.WriteLine("Hello, World!") ' Output with new line

Console.Write("Enter your name: ") ' Output without new line

VB.Net Input

Console.ReadLine() → Reads user input as a string.

If you need numeric input, convert it using Convert.ToInt32() or Convert.ToDouble().

Example:

Console.Write("Enter your age: ")

Dim age As Integer = Convert.ToInt32(Console.ReadLine())

Java Output

System.out.println() → Displays text and moves to the next line.

System.out.print() → Displays text without moving to the next line.

Example:

```
System.out.println("Hello, World!"); // Output with new line
```

```
System.out.print("Enter your name: "); // Output without new line
```

Java Input

Java uses the **Scanner** class from java.util package for input.

Steps:

1. Import the class: `import java.util.Scanner;`
2. Create a Scanner object: `Scanner input = new Scanner(System.in);`
3. Use methods like `nextLine()`, `nextInt()`, `nextDouble()`.

Example:

```
Scanner input = new Scanner(System.in);
```

```
System.out.print("Enter your age: ");
```

```
int age = input.nextInt();
```

Activity 1: Personalized Greeting Program

1. Create a console application in VB.NET and Java.
2. Display a welcome message.
3. Ask the user for:
 - o Name
 - o Course / Program (e.g., BSIT, BSCS, BSECE)
4. Display the output in the following format:

Welcome to Programming Class!

Name: [student's name]

Course: [student's course]

Activity 2: More Input and Output

1. Create a console program in VB.Net and Java.
2. Ask the user for:
 - a. Name
 - b. Year
 - c. Section
 - d. Address
 - e. Birthdate
 - f. Hobby
3. Display the data

Answer the following questions based on your personal experience in doing the activity.

1. While writing your VB.NET and Java programs, what differences did you notice between how the two languages handle input and output? Which one felt easier to use, and why?

2. What challenges did you encounter when asking the user for multiple inputs (like name, course, address, hobby)? How did you solve or overcome those challenges?

3. When running your program, how did it feel to see your own personal information displayed by the computer? Did it help you better understand the purpose of input and output in programming?

4. If you made mistakes (such as wrong data type conversion or syntax errors), how did you identify and fix them? What did you learn from that debugging process?

5. How can the skills you practiced in this activity (basic input/output) be useful when creating real-world applications like registration systems, login forms, or surveys? Give one concrete example.

Observation:

Conclusion:

NAME :		DATE :	
SCHEDULE :		RATING :	

Laboratory 2: Variables, Constants, and Operators

Status Check	Criteria	Poor 1	Fair 2	Good 3	Excellent 4
Advance <i>(100 pts. plus 5 extra pts.)</i>	Functionality	___ Not working	___ Several errors	___ Mostly work	___ Fully functional
	Design	___ Confusing or difficult to follow	___ Lacks clear structure.	___ Mostly organized	___ Content is well-arranged
On Time <i>(80 pts to 100 pts)</i>	Completeness	___ Incomplete Data	___ Partial Completion	___ Mostly complete	___ Fully complete
	Neatness of work	___ Messy and disorganized	___ Somewhat neat and good	___ Neat and organized	___ Exceptionally neat and organized
Late <i>(70 pts to 79 pts)</i>	Analytical Ability	___ Lack Analysis	___ Limited Analysis	___ Demonstrates good ability	___ Good analytical skills and ability

Objective:

Declare and use variables and constants in VB.NET and Java.

Apply arithmetic and relational operators to solve basic problems.

Develop a simple program that accepts user input, processes data, and displays results.

Introduction

Variables

A variable is a named storage location in memory that can hold a value.

In both VB.NET and Java, variables have data types (e.g., Integer, Double, String).

Example:

VB.Net: *Dim number As Integer = 10*

Java: *int number = 10;*

Constants

A constant is similar to a variable, but its value cannot be changed once assigned.

Used for fixed values like $\pi = 3.1416$ or conversion rates.

Example:

VB.Net: *Const PI As Double = 3.1416*

Java: *final double PI = 3.1416;*

Operators

Operators are symbols that perform operations on variables and values.

Arithmetic Operators: +, -, *, /, % (modulus).

Relational Operators: <, >, <=, >=, ==, <> (VB.NET), != (Java).

Logical Operators: And, Or, Not (VB.NET) | &&, ||, ! (Java).

VB.Net Example:

```
Module OperatorsExample
Sub Main()
    ' Input
    Console.WriteLine("Enter first number: ")
    Dim num1 As Integer = Convert.ToInt32(Console.ReadLine())

    Console.WriteLine("Enter second number: ")
    Dim num2 As Integer = Convert.ToInt32(Console.ReadLine())

    ' Arithmetic
    Console.WriteLine("Addition: " & (num1 + num2))
    Console.WriteLine("Subtraction: " & (num1 - num2))
    Console.WriteLine("Multiplication: " & (num1 * num2))
    Console.WriteLine("Division: " & (num1 / num2))
    Console.WriteLine("Modulus: " & (num1 Mod num2))

    ' Relational
    If num1 > num2 Then
        Console.WriteLine(num1 & " is greater than " & num2)
    ElseIf num1 < num2 Then
        Console.WriteLine(num1 & " is less than " & num2)
    Else
        Console.WriteLine(num1 & " is equal to " & num2)
    End If

    Console.ReadLine()
End Sub
End Module
```

Java Example:

```
import java.util.Scanner;

public class OperatorsExample {
    public static void main(String[] args) {
        Scanner input = new Scanner(System.in);

        // Input
        System.out.print("Enter first number: ");
        int num1 = input.nextInt();

        System.out.print("Enter second number: ");
        int num2 = input.nextInt();
```

```

// Arithmetic
System.out.println("Addition: " + (num1 + num2));
System.out.println("Subtraction: " + (num1 - num2));
System.out.println("Multiplication: " + (num1 * num2));
System.out.println("Division: " + (num1 / num2));
System.out.println("Modulus: " + (num1 % num2));

// Relational
if (num1 > num2) {
    System.out.println(num1 + " is greater than " + num2);
} else if (num1 < num2) {
    System.out.println(num1 + " is less than " + num2);
} else {
    System.out.println(num1 + " is equal to " + num2);
}
}
}

```

Activity 1: Arithmetic Operation

1. Create a console application in VB.Net and Java
2. Ask the user to following:
 - a. Employee name
 - b. Employee ID number
 - c. Working hours
 - d. Working rate
 - e. Tardiness
3. Compute for the employee's salary including the deduction of tax and tardiness
4. Display the employee's name, ID number and the total salary.

Activity 2: Temperature Comparison

1. Create a console program in VB.Net and Java
2. Ask the user to input room temperature
3. Determine if the temperature is below or above freezing point, or in boiling point
4. Display the result

Answer the following questions based on your personal experience in doing the activity.

1. In **Activity 1 (Arithmetic Operation)**, how did you decide which variables should be declared as integers, doubles, or constants? Did you encounter errors with data types, and how did you fix them?

-
-
2. When computing the employee's salary, what part of the calculation (basic pay, tax, or tardiness deduction) was most challenging to implement, and why?

-
-
-
-
3. In **Activity 2 (Temperature Comparison)**, how did using relational operators (<, >, =) help you determine if the temperature was below freezing point, at boiling point, or in between?

-
-
-
-
4. Did you encounter any logical errors (e.g., wrong conditions, incorrect formulas) while coding the salary or temperature program? How did you identify and correct those mistakes?

-
-
-
-
5. After completing both activities, how do you see the use of **variables, constants, and operators** being applied in real-life situations (e.g., payroll systems, weather monitoring, or billing)?
-
-
-
-

Observation:

Conclusion:

NAME :		DATE :	
SCHEDULE :		RATING :	

Laboratory 3: Control Structure – Conditional Statements

Status Check	Criteria	Poor 1	Fair 2	Good 3	Excellent 4
Advance (100 pts. plus 5 extra pts.)	Functionality	___ Not working	___ Several errors	___ Mostly work	___ Fully functional
	Design	___ Confusing or difficult to follow	___ Lacks clear structure.	___ Mostly organized	___ Content is well-arranged
On Time (80 pts to 100 pts)	Completeness	___ Incomplete Data	___ Partial Completion	___ Mostly complete	___ Fully complete
	Neatness of work	___ Messy and disorganized	___ Somewhat neat and good	___ Neat and organized	___ Exceptionally neat and organized
Late (70 pts to 79 pts)	Analytical Ability	___ Lack Analysis	___ Limited Analysis	___ Demonstrates good ability	___ Good analytical skills and ability

Objective:

Understand the concept of control structures in programming, specifically conditional statements (if, if-else, else-if, and switch/Select Case).

Apply conditional statements to solve problems that require decision-making in programs.

Develop simple applications in VB.NET and Java that use input, variables, operations, and conditional logic to produce meaningful results.

Analyze and test different input values to verify that conditions are working correctly and ensure the accuracy of program output.

Introduction

Control structures allow a program to make decisions and control the flow of execution. One of the control structures is condition statement

1. The If Statement

The if statement is the simplest form of decision-making.

It executes a block of code only if the condition is true.

If the condition is false, the program skips the block.

Example VB.Net:

```
Dim age As Integer = 20
If age >= 18 Then
    Console.WriteLine("You are an adult.")
End If
```

Example Java:

```
int age = 20;
if (age >= 18) {
    System.out.println("You are an adult.");
}
```

2. The If-Else Statement

Sometimes you want the program to do one thing if true, and another if false. This is done using if-else.

Example VB.Net

```
Dim score As Integer = 70
If score >= 75 Then
    Console.WriteLine("You passed!")
Else
    Console.WriteLine("You failed.")
End If
```

Example Java:

```
int score = 70;
if (score >= 75) {
    System.out.println("You passed!");
} else {
    System.out.println("You failed.");
}
```

3. The Elseif/Else-If Statement (Multiple Conditions)

If there are more than two possible outcomes, we use Elseif (VB.NET) or else if (Java). This checks conditions in order from top to bottom. Only the first true condition is executed.

Example VB.Net:

```
Dim grade As Integer = 85
If grade >= 90 Then
    Console.WriteLine("Grade: A")
Elseif grade >= 80 Then
    Console.WriteLine("Grade: B")
Elseif grade >= 70 Then
    Console.WriteLine("Grade: C")
Elseif grade >= 60 Then
    Console.WriteLine("Grade: D")
Else
    Console.WriteLine("Grade: F")
End If
```

Example Java:

```
int grade = 85;
if (grade >= 90) {
    System.out.println("Grade: A");
} else if (grade >= 80) {
    System.out.println("Grade: B");
} else if (grade >= 70) {
    System.out.println("Grade: C");
}
```

```

    } else if (grade >= 60) {
        System.out.println("Grade: D");
    } else {
        System.out.println("Grade: F");
    }
}

```

4. Case/Switch Statement

When there are many options, using multiple else-if conditions can be messy. Instead, we can use Select Case (VB.NET) or switch (Java). These are useful when comparing a single value against several possible values.

Example VB.Net:

```

Dim day As Integer = 3
Select Case day
    Case 1
        Console.WriteLine("Monday")
    Case 2
        Console.WriteLine("Tuesday")
    Case 3
        Console.WriteLine("Wednesday")
    Case 4
        Console.WriteLine("Thursday")
    Case 5
        Console.WriteLine("Friday")
    Case 6
        Console.WriteLine("Saturday")
    Case 7
        Console.WriteLine("Sunday")
    Case Else
        Console.WriteLine("Invalid day")
End Select

```

Example Java:

```

int day = 3;
switch (day) {
    case 1:
        System.out.println("Monday");
        break;
    case 2:
        System.out.println("Tuesday");
        break;
    case 3:
        System.out.println("Wednesday");
        break;
    case 4:
        System.out.println("Thursday");

```

```

        break;
    case 5:
        System.out.println("Friday");
        break;
    case 6:
        System.out.println("Saturday");
        break;
    case 7:
        System.out.println("Sunday");
        break;
    default:
        System.out.println("Invalid day");
}

```

Activity 1: Simple Calculator

1. Create a console program in VB.Net and Java
2. Ask the user to input two numbers.
3. Ask the user to choose an operation (+, -, *, /, %).
4. Use if-else (or switch) to perform the selected operation.
5. Display the result.

Activity 2: Body Mass Index (BMI) Calculator

1. Create a console program in VB.Net and Java
2. Ask the user for:
 - a. Name
 - b. Age
 - c. Weight in kilograms
 - d. Height in meters
3. Compute the **BMI = weight / (height × height)**.
4. Use conditions to determine the category:
 - BMI < 18.5 → Underweight
 - 18.5–24.9 → Normal
 - 25–29.9 → Overweight
 - 30 and above → Obese
5. Display the BMI and category.

Answer the following questions based on your personal experience in doing the activity.

1. In the Simple Calculator, what difficulties did you face when handling different operations (e.g., addition vs. division), and how did you solve them?

-
-
2. Did you encounter any errors when dividing numbers, such as division by zero? How did you prevent or fix this issue?

-
-
-
-
3. In the BMI Calculator, how did you check if your formula and conditional statements were giving the correct category (Underweight, Normal, Overweight, Obese)?

-
-
-
-
4. Which program did you find easier to test and debug: the Calculator or the BMI Calculator? Why

-
-
-
-
5. After completing both activities, how do you see the importance of variables, operators, and conditional statements in solving real-world problems?
-
-
-
-

Observation:

Conclusion:

NAME :		DATE :	
SCHEDULE :		RATING :	

Laboratory 4: Control Structure – Looping Structure

Status Check	Criteria	Poor 1	Fair 2	Good 3	Excellent 4
Advance <i>(100 pts. plus 5 extra pts.)</i>	Functionality	___ Not working	___ Several errors	___ Mostly work	___ Fully functional
	Design	___ Confusing or difficult to follow	___ Lacks clear structure.	___ Mostly organized	___ Content is well-arranged
On Time <i>(80 pts to 100 pts)</i>	Completeness	___ Incomplete Data	___ Partial Completion	___ Mostly complete	___ Fully complete
	Neatness of work	___ Messy and disorganized	___ Somewhat neat and good	___ Neat and organized	___ Exceptionally neat and organized
Late <i>(70 pts to 79 pts)</i>	Analytical Ability	___ Lack Analysis	___ Limited Analysis	___ Demonstrates good ability	___ Good analytical skills and ability

Objective:

- Understand and apply different looping constructs (for, while, and do-while) in VB.NET and Java to solve repetitive tasks.
- Develop problem-solving skills by creating interactive programs that require repetition and decision-making.
- Demonstrate the integration of loops with input/output operations, variables, and conditional statements to build practical, real-world console applications.

Introduction

What are Loops?

- Loops are control structures that allow us to **repeat a block of code multiple times** without writing it again.

There are 3 common types of loops:

1. **For Loop** – used when the number of repetitions is known.
2. **While Loop** – used when the condition must be true before entering the loop.
3. **Do-While Loop** – executes at least once before checking the condition.

For Loop

Best when the number of iterations is fixed.

Example VB.Net:

```
For i As Integer = 1 To 5
    Console.WriteLine("Number: " & i)
Next
```

Example Java:

```
for (int i = 1; i <= 5; i++) {
    System.out.println("Number: " + i);
}
```

While Loop

Executes **while** the condition is true.

Example VB.Net

```
Dim i As Integer = 1
While i <= 5
    Console.WriteLine("Number: " & i)
    i += 1
End While
```

Example Java:

```
int i = 1;
while (i <= 5) {
    System.out.println("Number: " + i);
    i++;
}
```

Do-While Loop

Executes the block **at least once**, then checks the condition.

Example VB.Net:

```
Dim i As Integer = 1
Do
    Console.WriteLine("Number: " & i)
    i += 1
Loop While i <= 5
```

Example Java:

```
int i = 1;
do {
    System.out.println("Number: " + i);
    i++;
} while (i <= 5);
```

Task Example: Multiplication Table

VB.Net

```
Console.Write("Enter a number: ")
Dim num As Integer = Convert.ToInt32(Console.ReadLine())

For i As Integer = 1 To 10
    Console.WriteLine(num & " x " & i & " = " & (num * i))
Next
```

Java

```
import java.util.Scanner;
public class MultiplicationTable {
    public static void main(String[] args) {
```

```

Scanner input = new Scanner(System.in);
System.out.print("Enter a number: ");
int num = input.nextInt();

for (int i = 1; i <= 10; i++) {
    System.out.println(num + " x " + i + " = " + (num * i));
}
}
}

```

Activity 1: Guessing Game

1. Create a console program in Java
2. Ask the user to input two numbers (for lowest and highest).
3. Generate a random number between the lowest and highest.
4. Ask the user to guess the number until correct.
5. Provide hints: "Too High" or "Too Low"
6. Count the number of attempts

Activity 2: Simple ATM Simulation

1. Create a console program in VB.Net
2. User starts with a balance (e.g., 1000).
3. Show a menu (Deposit, Withdraw, Check Balance, Exit).
4. Use a loop to repeat until the user chooses Exit.
5. Apply conditionals for invalid amounts (e.g., withdraw more than balance).

Activity 3: Simple Library System

1. Create a console program in VB.Net.
2. Show menu options: Borrow Book, Return Book, Check Available Books, Exit.
3. Ask the student for:
 - a. Name
 - b. Address
 - c. Contact details

If choosing borrow or return books

4. Use a loop to repeat until Exit is chosen.
5. Update the number of available books after each transaction.

Answer the following questions based on your personal experience in doing the activity.

1. In the **Guessing Game**, how did you feel about using a loop to keep asking for input until the user guessed correctly? What challenges did you face when implementing the "Too High" or "Too Low" hints?

2. For the **ATM Simulation**, which part was more difficult: handling deposits/withdrawals with conditionals or keeping the program running in a loop until “Exit”? Explain your experience.

3. In the **Library System**, how did you make sure that the number of available books was correctly updated after each borrow or return transaction? Did you encounter any logic errors?

4. Between the three activities, which one required the most debugging? What errors did you encounter, and how did you solve them?

5. After finishing these activities, how do you see the importance of combining **loops, conditionals, and user input** in building interactive programs?

Observation:

Conclusion:

NAME :		DATE :	
SCHEDULE :		RATING :	

Laboratory 5: Nested Control Structures

Status Check	Criteria	Poor 1	Fair 2	Good 3	Excellent 4
Advance <i>(100 pts. plus 5 extra pts.)</i>	Functionality	___ Not working	___ Several errors	___ Mostly work	___ Fully functional
	Design	___ Confusing or difficult to follow	___ Lacks clear structure.	___ Mostly organized	___ Content is well-arranged
On Time <i>(80 pts to 100 pts)</i>	Completeness	___ Incomplete Data	___ Partial Completion	___ Mostly complete	___ Fully complete
	Neatness of work	___ Messy and disorganized	___ Somewhat neat and good	___ Neat and organized	___ Exceptionally neat and organized
Late <i>(70 pts to 79 pts)</i>	Analytical Ability	___ Lack Analysis	___ Limited Analysis	___ Demonstrates good ability	___ Good analytical skills and ability

Objective:

Understand the concept of nested control structures by combining loops and conditional statements in solving more complex problems.

Develop logical thinking and problem-solving skills by applying multiple layers of decision-making (e.g., conditions inside loops, loops inside loops).

Implement real-world scenarios using nested control structures in VB.NET and Java.

Enhance programming efficiency and readability by organizing code through proper use of nested if-else, switch/case, and iterative structures.

Introduction

A nested control structure means putting one control structure (loop or condition) inside another.

You can nest if-else inside a loop.

You can nest a loop inside another loop.

Or a loop with multiple conditions.

This is useful when we need **multiple layers of logic**.

Example: In a Password Validation with Strength Check system, we may need:

1. A **loop** to allow multiple transactions.
2. A **nested condition** to check valid inputs.
3. A **nested loop** for checking characters.

VB.Net Code:

```

Module PasswordCheck
    Sub Main()
        Dim attempts As Integer = 0

        While attempts < 3
            Console.WriteLine("Enter a password: ")
            Dim password As String = Console.ReadLine()

            ' Check if length is valid

```

```

If password.Length >= 6 Then
    Dim hasNumber As Boolean = False

    ' Loop through each character
    For Each ch As Char In password
        If Char.IsDigit(ch) Then
            hasNumber = True
            Exit For
        End If
    Next

    If hasNumber Then
        Console.WriteLine("Password accepted!")
        Exit While
    Else
        Console.WriteLine("Password must contain at least one number.")
    End If
Else
    Console.WriteLine("Password too short.")
End If

attempts += 1
End While

If attempts = 3 Then
    Console.WriteLine("Too many failed attempts. Access denied.")
End If
End Sub
End Module

```

Java Code:

```

import java.util.Scanner;

public class PasswordCheck {
    public static void main(String[] args) {
        Scanner sc = new Scanner(System.in);
        int attempts = 0;

        while (attempts < 3) {
            System.out.print("Enter a password: ");
            String password = sc.nextLine();

            if (password.length() >= 6) {
                boolean hasNumber = false;
                for (int i = 0; i < password.length(); i++) {
                    if (Character.isDigit(password.charAt(i))) {
                        hasNumber = true;
                    }
                }
            }
        }
    }
}

```

```

        break;
    }
}

if (hasNumber) {
    System.out.println("Password accepted!");
    break;
} else {
    System.out.println("Password must contain at least one number.");
}
} else {
    System.out.println("Password too short.");
}
attempts++;
}

if (attempts == 3) {
    System.out.println("Too many failed attempts. Access denied.");
}
}
}

```

Activity 1: Bank Loan Payment

1. Create a program in **Java** to simulate loan repayment
2. Ask the user to input Loan Amount, Interest Rate and Number of Months for payment.
3. Compute the total payment per month with interest.
4. Ask if payment is late each month. If late → add penalty.
5. Display month number, payment status, and remaining balance after each loop.
6. End program when loan is fully paid, then show total months taken.

Activity 2: Student Grades with Classification

1. Create a program in **VB.Net** to classify student grades.
2. Ask the user for the number of students.
3. Input each student's score (0–100).
 - 90–100 → A
 - 80–89 → B
 - 70–79 → C
 - 60–69 → D
 - Below 60 → F
4. At the end, display how many students got A, B, C, D, or F.

Answer the following questions based on your personal experience in doing the activity.

1. In the **Bank Loan Simulation**, how did the loop and conditional statements help you manage the monthly updates of interest, payments, and penalties?

2. What challenges did you encounter when checking for **late payments** and applying penalties? How did you solve them?

3. In the **Student Grades Classification**, how did you structure your **if-else conditions (or Select Case)** to ensure correct grade assignments?

4. Which was easier to implement: the **loan repayment loop with conditions** or the **grade classification**? Why?

5. If you could improve these programs, what **additional features** would you add (e.g., summary of total interest paid for loans, or percentage of students in each grade category)?

Observation:

Conclusion:

NAME :		DATE :	
SCHEDULE :		RATING :	

Laboratory 6: Arrays and Functions/Methods

Status Check	Criteria	Poor 1	Fair 2	Good 3	Excellent 4
Advance <i>(100 pts. plus 5 extra pts.)</i>	Functionality	___ Not working	___ Several errors	___ Mostly work	___ Fully functional
	Design	___ Confusing or difficult to follow	___ Lacks clear structure.	___ Mostly organized	___ Content is well-arranged
On Time <i>(80 pts to 100 pts)</i>	Completeness	___ Incomplete Data	___ Partial Completion	___ Mostly complete	___ Fully complete
	Neatness of work	___ Messy and disorganized	___ Somewhat neat and good	___ Neat and organized	___ Exceptionally neat and organized
Late <i>(70 pts to 79 pts)</i>	Analytical Ability	___ Lack Analysis	___ Limited Analysis	___ Demonstrates good ability	___ Good analytical skills and ability

Objective:

Understand and apply arrays in VB.NET and Java to efficiently store, manage, and process multiple data values under a single variable name.

Develop modular programming skills by implementing functions (methods) and subroutines to perform specific tasks such as computing sums, averages, maximum, and minimum values.

Differentiate parameter passing methods (ByVal vs ByRef in VB.NET) and demonstrate how they affect variables inside and outside a function.

Apply arrays and functions to real-world problems to strengthen problem-solving and logical thinking skills.

Introduction

Arrays

An **array** is a collection of variables of the same type, stored under one name.

Example: instead of having 5 different variables (n1, n2, n3, n4, n5), you can use an array: numbers(0..4) in VB.Net or numbers[0..4] in Java.

VB.Net:

```
Dim numbers(4) As Integer ' Array of 5 elements
numbers(0) = 10
numbers(1) = 20
```

Java:

```
int[] numbers = new int[5]; // Array of 5 elements
numbers[0] = 10;
numbers[1] = 20;
```

Functions and Methods

A **function/method** is a reusable block of code that performs a specific task.

- Functions return a value.
- Subroutines (VB.Net) or void methods (Java) do not return a value.

VB.Net Example (Function to compute sum):

```
Function GetSum(arr() As Integer) As Integer
    Dim sum As Integer = 0
    For Each num In arr
        sum += num
    Next
    Return sum
End Function
```

Java Example (Method to compute sum):

```
static int getSum(int[] arr) {
    int sum = 0;
    for (int num : arr) {
        sum += num;
    }
    return sum;
}
```

ByVal vs ByRef (VB.Net only)

- **ByVal** → passes a **copy** of the variable (original value does not change).
- **ByRef** → passes the **actual variable** (changes inside the function affect the original value).

Example:

```
Sub SquareByVal(ByVal x As Integer)
    x = x * x
End Sub
```

```
Sub SquareByRef(ByRef x As Integer)
    x = x * x
End Sub
```

```
Dim num As Integer = 5
SquareByVal(num)
Console.WriteLine(num) ' Output: 5 (unchanged)
SquareByRef(num)
Console.WriteLine(num) ' Output: 25 (changed)
```

Activity 1: Bank Loan Payment Simulation

1. Create a program in **Java** to simulate loan repayment.
2. Ask the user to input the Loan Amount, Interest Rate and Number of Months.
3. Compute for the total monthly payment with interest.
4. Ask if payment is late each month. If late → add penalty.
5. Display month number, payment status, and remaining balance after each loop.
6. End program when loan is fully paid, then show total months taken.

Activity 2: 5x5 Tic-Tac-Toe Game

1. Create a program in Java (or VB.NET) to simulate a **5x5 Tic-Tac-Toe game**.
2. Use a **5x5 character array** to represent the game board.
3. Initialize the board with numbers (1–25) to indicate available positions.
4. Create a function to **display the board** after every turn.
5. Let **Player 1 (X)** and **Player 2 (O)** take turns choosing a position (1–25).
6. Write a function to **update the board** with the chosen move.
 - If the cell is already taken, prompt the player to choose again.
7. After each move, check if a player has **3 in a row**:
 - Same symbol across any row.
 - Same symbol in any column.
 - Same symbol along the two diagonals.
8. If a winner is found, display the winner and end the game.
9. If all 25 cells are filled with no winner, display **"It's a draw!"**.

Answer the following questions based on your personal experience in doing the activity.

1. While creating the Bank Loan Payment Simulation, what challenges did you face in computing monthly payments and updating the remaining balance? How did you solve them?

2. How did using functions/methods make your program more organized and easier to debug compared to writing all logic inside main()?

3. In the 5x5 Tic-Tac-Toe Game, how did you handle checking whether a position was already taken? What error or bug did you encounter while updating the board?

4. Which part of programming the 5x5 Tic-Tac-Toe was the most difficult—displaying the board, updating moves, or checking for a winner? Why?

5. Compare your experience in both activities: Which program did you find more engaging (Loan Payment or Tic-Tac-Toe), and what new programming concepts did you learn from it?

Observation:

Conclusion:

NAME :		DATE :	
SCHEDULE :		RATING :	

Laboratory 7: Object-Oriented Programming – Classes, Encapsulation, and Inheritance

Status Check	Criteria	Poor 1	Fair 2	Good 3	Excellent 4
Advance <i>(100 pts. plus 5 extra pts.)</i>	Functionality	___ Not working	___ Several errors	___ Mostly work	___ Fully functional
	Design	___ Confusing or difficult to follow	___ Lacks clear structure.	___ Mostly organized	___ Content is well-arranged
On Time <i>(80 pts to 100 pts)</i>	Completeness	___ Incomplete Data	___ Partial Completion	___ Mostly complete	___ Fully complete
	Neatness of work	___ Messy and disorganized	___ Somewhat neat and good	___ Neat and organized	___ Exceptionally neat and organized
Late <i>(70 pts to 79 pts)</i>	Analytical Ability	___ Lack Analysis	___ Limited Analysis	___ Demonstrates good ability	___ Good analytical skills and ability

Objective:

Understand the fundamentals of Object-Oriented Programming (OOP): classes, objects, properties, and methods.

Apply encapsulation by using private fields and public methods (getters/setters) to protect data.

Demonstrate inheritance by creating base and derived classes, showing code reuse and extension.

Develop problem-solving skills by creating real-world simulations using OOP concepts.

Introduction

Object-Oriented Programming (OOP) is a paradigm that organizes code around objects rather than just functions and logic.

Classes and Objects

- A class is a blueprint or template for creating objects.
- An object is an instance of a class with its own data and behavior.

VB.NET Example (Student Class):

```
Public Class Student
```

```
Public Property Name As String
```

```
Public Property Age As Integer
```

```
Public Property Course As String
```

```
Public Sub DisplayInfo()
```

```
Console.WriteLine("Name: " & Name & ", Age: " & Age & ", Course: " & Course)
```

```
End Sub
```

```
Public Function IsAdult() As Boolean
```

```
Return Age >= 18
```

```
End Function
```

```
End Class
```

Java Example (Student Class):

```
class Student {  
    String name;  
    int age;  
    String course;  
  
    void displayInfo() {  
        System.out.println("Name: " + name + ", Age: " + age + ", Course: " + course);  
    }  
  
    boolean isAdult() {  
        return age >= 18;  
    }  
}
```

Encapsulation

- Encapsulation means hiding the internal details of a class and only exposing necessary parts.
- Achieved using private variables and public methods (getters/setters).

Example Java:

```
class Account {  
    private double balance;  
  
    public void deposit(double amount) {  
        balance += amount;  
    }  
  
    public double getBalance() {  
        return balance;  
    }  
}
```

Example VB.Net:

```
class Account {  
    private double balance;  
  
    public void deposit(double amount) {  
        balance += amount;  
    }  
  
    public double getBalance() {  
        return balance;  
    }  
}
```

Inheritance

- Inheritance allows a class (child/subclass) to reuse and extend another class (parent/base class).
- Example: A Rectangle is a Shape.

Example VB.Net:

```
Public Class Shape
    Public Overridable Function Area() As Double
        Return 0
    End Function
End Class

Public Class Rectangle
    Inherits Shape
    Public Width As Double
    Public Height As Double

    Public Sub New(w As Double, h As Double)
        Width = w
        Height = h
    End Sub

    Public Overrides Function Area() As Double
        Return Width * Height
    End Function
End Class
```

Example Java:

```
class Shape {
    public double area() {
        return 0;
    }
}

class Rectangle extends Shape {
    double width, height;

    Rectangle(double w, double h) {
        width = w;
        height = h;
    }

    public double area() {
        return width * height;
    }
}
```

Polymorphism

- Polymorphism means one interface, many implementations.
- Example: A Shape reference can point to either a Rectangle or Circle, and call their Area() method without knowing the exact type.

Activity 1: Animal Sound

1. Create a program in Java.
2. The program asks the user how many animals they want to enter.
3. For each entry, the user types an animal type (Dog, Cat, Bird).
4. The program creates the correct object (Dog, Cat, Bird) and stores it in a list/array.
5. After input, the program loops through all animals and calls MakeSound().
6. If the user enters an unknown animal, display: *"Animal not recognized."*

Answer the following questions based on your personal experience in doing the activity.

1. While creating classes and objects, what challenges did you face in deciding which properties and methods to include?

2. How did encapsulation (using getters and setters) help you control access to your class data compared to using public variables?

3. When applying inheritance, how did it feel to reuse and extend code from the parent class instead of writing everything from scratch?

4. Looking back at your code, how would you improve or expand the program (e.g., adding more animals, or creating new shapes) while still using OOP principles?

Observation:

Conclusion:

NAME :		DATE :	
SCHEDULE :		RATING :	

Laboratory 8: VB.NET Windows Forms

Status Check	Criteria	Poor 1	Fair 2	Good 3	Excellent 4
Advance <i>(100 pts. plus 5 extra pts.)</i>	Functionality	___ Not working	___ Several errors	___ Mostly work	___ Fully functional
	Design	___ Confusing or difficult to follow	___ Lacks clear structure.	___ Mostly organized	___ Content is well-arranged
On Time <i>(80 pts to 100 pts)</i>	Completeness	___ Incomplete Data	___ Partial Completion	___ Mostly complete	___ Fully complete
	Neatness of work	___ Messy and disorganized	___ Somewhat neat and good	___ Neat and organized	___ Exceptionally neat and organized
Late <i>(70 pts to 79 pts)</i>	Analytical Ability	___ Lack Analysis	___ Limited Analysis	___ Demonstrates good ability	___ Good analytical skills and ability

Objective:

Transition from console-based programming to GUI programming, understanding the differences in input/output handling.

Design and implement a graphical user interface using Windows Forms (VB.NET)

Apply event-driven programming to handle user interactions such as button clicks and input validation.

Develop problem-solving skills by creating a functional calculator with proper error handling.

Understand the separation of UI and logic, improving code organization and maintainability in GUI applications.

Introduction

GUI programs are event-driven:

- Instead of executing line by line only, the program waits for user actions (events like button clicks, typing text, or selecting from a menu).
- Each event triggers a specific block of code (called an event handler).

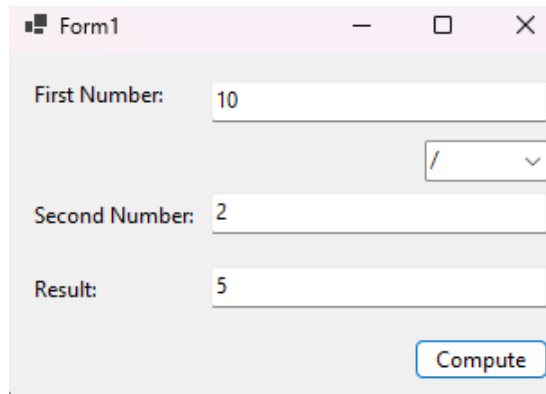
Example:

```
Public Class CalculatorForm
    Private Sub btnAdd_Click(sender As Object, e As EventArgs) Handles btnAdd.Click
        Dim num1 As Double = Convert.ToDouble(txtNum1.Text)
        Dim num2 As Double = Convert.ToDouble(txtNum2.Text)
        Dim result As Double = num1 + num2
        lblResult.Text = "Result: " & result.ToString()
    End Sub
End Class
```

- txtNum1, txtNum2 → textboxes for input.
- btnAdd → button for addition.
- lblResult → label to display result.
- When the button is clicked, the **event handler (btnAdd_Click)** runs.

Activity 1: Simple Calculator

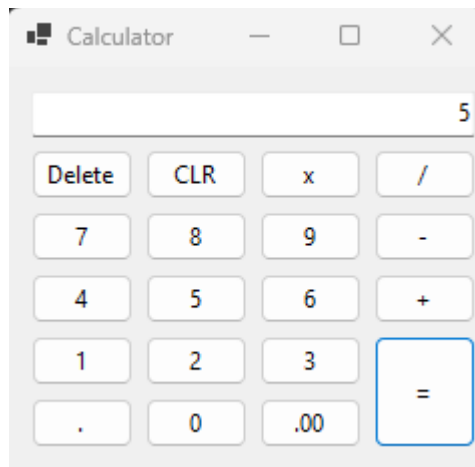
1. Create a Windows Form (VB.NET)
2. Add two TextBoxes (for input), Combobox for operation, Button for compute and one Label to display the result.
3. Code the button click events to perform the correct operation.
4. If division by zero is attempted, display an error message ("Cannot divide by zero!").
5. Run the program and test all four operations.



The screenshot shows a Windows Form titled "Form1" with a light gray background. It contains three text boxes: "First Number:" with the value "10", "Second Number:" with the value "2", and "Result:" with the value "5". Between the first and second number boxes is a dropdown menu showing the division operator "/". At the bottom right, there is a button labeled "Compute".

Activity 2: Calculator

1. Create a Windows Form (VB.NET) or Swing/JavaFX app (Java).
2. Design the form to look like a real calculator, including:
 - Buttons: 0–9 digits, Add (+), Subtract (−), Multiply (×), Divide (÷), Clear (C), and Equals (=).
 - TextBox or Label: To display input and result.
3. Arrange the buttons in a grid layout similar to a standard calculator.
4. Implement button functionality:
 - Clicking a digit button appends the number to the display.
 - Clicking an operator button stores the current number and operator.
 - Clicking Equals (=) performs the calculation and displays the result.
 - Clicking Clear (C) resets the display.
5. Handle exceptions or invalid inputs (e.g., division by zero) with an error message.



The screenshot shows a standard Windows calculator application window titled "Calculator". It features a display showing the number "5". Below the display is a grid of buttons: "Delete", "CLR", "x", and "/" in the first row; "7", "8", "9", and "-" in the second row; "4", "5", "6", and "+" in the third row; "1", "2", "3", and "=" in the fourth row; and ".", "0", ".00", and "=" in the fifth row. The "=" button in the fourth row is highlighted with a blue border.

Answer the following questions based on your personal experience in doing the activity.

1. How did designing a GUI (buttons, textboxes, labels) change your approach compared to console-based calculators?

2. What challenges did you encounter when arranging the buttons in a grid layout, and how did you solve them?

3. How did you implement the logic to handle multiple operations and ensure the correct result is displayed?

4. How did you handle invalid inputs (like dividing by zero) or unexpected user actions?

5. Which part of event-driven programming did you find most interesting or different from sequential programming?

6. If you were to add new features (e.g., square root, memory functions, or percentage), how would you modify your program?

7. How does separating the GUI design from the calculation logic help in maintaining or updating your program?

Observation:

Conclusion:

NAME :		DATE :	
SCHEDULE :		RATING :	

Laboratory 9: Registration / Login System with SQL

Status Check	Criteria	Poor 1	Fair 2	Good 3	Excellent 4
Advance <i>(100 pts. plus 5 extra pts.)</i>	Functionality	___ Not working	___ Several errors	___ Mostly work	___ Fully functional
	Design	___ Confusing or difficult to follow	___ Lacks clear structure.	___ Mostly organized	___ Content is well-arranged
On Time <i>(80 pts to 100 pts)</i>	Completeness	___ Incomplete Data	___ Partial Completion	___ Mostly complete	___ Fully complete
	Neatness of work	___ Messy and disorganized	___ Somewhat neat and good	___ Neat and organized	___ Exceptionally neat and organized
Late <i>(70 pts to 79 pts)</i>	Analytical Ability	___ Lack Analysis	___ Limited Analysis	___ Demonstrates good ability	___ Good analytical skills and ability

Objective:

To design and implement a Registration Form in VB.NET using Windows Forms controls such as labels, textboxes, and buttons.

To apply SQL commands (INSERT and SELECT) in saving and retrieving student registration data from a database.

To develop a Login Form that validates user credentials through database queries.

To demonstrate proper error handling and secure data entry using parameterized queries and password protection.

Introduction

Registration Form

Registration Form contains these input fields:

Name – full name of the student

Year – academic year (e.g., 1st Year, 2nd Year, etc.)

Course – program or strand (e.g., BSIT, BSCS)

Section – class section (e.g., IT-201A)

Student Number – unique ID for each student

Password – chosen password for login security

It also has two buttons:

Back – to return to the previous form/window

Save – to store the entered registration data

Tools Used in Windows Form:

Label → For captions ("Name:", "Year:", etc.)

TextBox → For input fields

Button → For actions like *Save* and *Back*

Form → Container window

PasswordChar property in TextBox → For hidden password input

What is SQL?

SQL (Structured Query Language) is a standard language used to manage and manipulate databases. It is used to insert, update, delete, and retrieve data from relational databases like MySQL, SQL Server, or SQLite.

Common SQL commands:

- INSERT → Add new data
- SELECT → Retrieve data
- UPDATE → Modify data
- DELETE → Remove data

How to Save Data from Registration

When the student clicks the Save button, the program will run an INSERT SQL query.

Example Query:

```
INSERT INTO Students (Name, Year, Course, Section, StudentNumber, Password)
VALUES ('Juan Dela Cruz', '2nd Year', 'BSIT', 'IT-201A', '2025-00123', 'mypassword');
```

Example VB.Net

```
Imports System.Data.SqlClient
```

```
Public Class RegistrationForm
```

```
Dim con As New SqlConnection("Data Source=.;Initial Catalog=SchoolDB;Integrated
Security=True")
```

```
Private Sub btnSave_Click(sender As Object, e As EventArgs) Handles btnSave.Click
```

```
Try
```

```
con.Open()
```

```
Dim query As String = "INSERT INTO Students (Name, Year, Course, Section,
StudentNumber, Password)
```

```
VALUES (@Name, @Year, @Course, @Section, @StudentNumber,
@Password)"
```

```
Dim cmd As New SqlCommand(query, con)
```

```
cmd.Parameters.AddWithValue("@Name", txtName.Text)
```

```
cmd.Parameters.AddWithValue("@Year", txtYear.Text)
```

```
cmd.Parameters.AddWithValue("@Course", txtCourse.Text)
```

```
cmd.Parameters.AddWithValue("@Section", txtSection.Text)
```

```
cmd.Parameters.AddWithValue("@StudentNumber", txtStudentNumber.Text)
```

```
cmd.Parameters.AddWithValue("@Password", txtPassword.Text)
```

```
cmd.ExecuteNonQuery()
```

```
MessageBox.Show("Registration successful!")
```

```

        Catch ex As Exception
            MessageBox.Show(ex.Message)
        Finally
            con.Close()
        End Try
    End Sub
End Class

```

Retrieving Data for Login

To verify login, use **SELECT**.

Example Query:

```

SELECT * FROM Students
WHERE StudentNumber = '2025-00123' AND Password = 'mypassword';

```

Activity: Login Form

1. Create a Login Form with:
 - TextBox for *Student Number*
 - TextBox for *Password* (set PasswordChar = *)
 - Login Button
2. When Login is clicked:
 - Use SELECT query to check database.
 - If found → Display "Login Successful!"
 - If not found → Display "Invalid Login!"
3. After login, open a new form showing the student's name and details retrieved from the database.

Answer the following questions based on your personal experience in doing the activity.

1. Why is the Student Number required to be unique, and what might happen if duplicates are allowed?

2. Why is it important to use the PasswordChar property for the password textbox?

3. What happens if you forget to close the database connection after saving data?

-
-
4. If the database table Students has an extra column (like Email), how would you modify the INSERT SQL query and the VB.NET code?

-
-
-
-
5. Suppose the user clicks Save but one of the textboxes is empty—how can you handle this situation in VB.NET before running the SQL query?

-
-
-
-
6. What error might occur if you try to save a record with the same Student Number twice, and how would you handle it in code?

-
-
-
-
7. What SQL query is used to check if the entered Student Number and Password exist in the database?

-
-
-
-
8. Why do we compare both Student Number and Password instead of just one?

-
-
-
9. What message is displayed if the login details do not match?

10. What security risks exist if you save passwords in plain text, and what would be a better approach?

11. What would happen if the login query is written without WHERE clause?

12. What would you change in the query if you want to retrieve and display the student's Name and Course after login?

Observation:

Conclusion:
